

1. Analyze two GenAI programs:

Meta Muse Spark 1.1 | Thinking

Google Gemini 3.1 | Pro extended thinking

2. How you accessed them:

<https://www.meta.ai/> | Meta Muse Spark 1.1 | Thinking

<https://gemini.google.com/app> | Google Gemini 3.1 | Pro extended thinking

3. Prompt used

The program executes Sample Test File and Grader's Test File and uses a MaxHeap as a means of implementing a priority queue using a list-based implementation and does not include pre-existing heap modules.

Requirements:

- The Program must be written in C, not C++
- You will use a MaxHeap as a means of implementing a priority queue. A priority queue is a queue where emails can shift towards the front of the queue based on a priority status.
- You must implement a MaxHeap using a list-based implementation. Then use that MaxHeap can handle all your email prioritizing for the CEO.
- You must create functions from scratch. Do not include pre-existing heap modules

Test File format:

| Command | Description |

| :--- | :--- |

| EMAIL <sender category>, <subject line>, <date> | The emails in the CEO's Inbox should be placed in queue based on their sender category and date. The sender categories and priority to be read are as follows:
• Boss – read first
• Subordinate – read next
• Peer – read next
• ImportantPerson – read next
• OtherPerson – read last

If there is more than one from a sender, then the newest email (not the oldest) should be read first. I discovered this trick while a manager at Sprint.

EMAIL is followed by space. The rest of the fields are delimited.
Assume <sender category> is one of the five strings listed above.
Assume <subject line> is a string which may contain spaces, but not commas
Assume <date> is in the format: MM-DD-YYYY |

| NEXT | Next email for the CEO to read. Display the information on the terminal in the following format:
Sender:
Subject:
Date: |

| READ | CEO has read email and dealt with it |

| COUNT | Display current count of unread emails |

• The grader will test your code with a different test file. The test file will begin with the emails in the CEOs Inbox, followed by the commands the CEO will use to read her Inbox. You can assume a given command will be formatted as shown, but the order and number of commands may vary. For example, be able to handle a file that does NEXT or READ when there are no emails, two NEXTs in a row without an intervening READ (output should be the same for both NEXTs), two READs in a row without an intervening NEXT (highest priority email deleted from queue without displaying it).

Sample Test File:

EMAIL Peer,Can you help me on this?,12-01-2024

EMAIL OtherPerson,Try our product,12-19-2024

EMAIL Boss,Important,12-20-2024

EMAIL Subordinate,How do I handle this?,12-25-2024

EMAIL ImportantPerson,Health Insurance Enrollment,12-31-2024

EMAIL Boss,Never Mind,01-03-2025

COUNT

NEXT

READ

NEXT

READ

COUNT

Display:

There are 6 emails to read.

Next email:

Sender: Boss

Subject: Never Mind

Date: 01-03-2025

Next email:

Sender: Boss

Subject: Important

Date: 12-20-2024

There are 4 emails to read.

Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the "GenAI Code Comparison" lecture.

The two GenAI examples include one other difference.

- What is the difference?

Meta Muse Spark 1.1 | Pointer array

Google Gemini 3.1 | Value array

These are just two ways to implement an array for a maxheap

Analyze the difference in terms of:

- Execution time

Because Meta Muse uses a pointer array, it only swaps one 8 bit pointer so heapifyup and heapifydown is much better in terms of execution time. Because Google Gemini 3.1 has to copy the whole 300 byte struct every time it swaps, meaning that execution time is closer to $O(n)$ compared to $O(\log n)$. Meaning that Meta Muse Spark code is more efficient in terms of execution time

- Space complexity

Google Gemini 3.1 is better in terms of space complexity because it keeps everything in an array while Meta Muse 1.1 keeps everything fragmented using malloc. Meaning that values aren't kept together and take up more relative space.

- Maintainability

Because Meta Muse does everything so precisely, that means that there are higher chances of memory leaks, double frees or use frees after code changes. Because Google Gemini uses an array that means that there are moving parts when things go wrong.

3. Which of the two GenAI code examples would you select? Why?

I personally would use Google Gemini's code because despite it being slower, being able to maintain the code despite taking up more space and more time is a worthwhile trade off.

4. How would you improve the following for the one you selected

- Correctness

Added a trim() pass on sender/subject/date after splitting, so " Boss" or "Boss " still matches "Boss" in getCatPriority() instead of silently falling through to priority 0

Malformed EMAIL lines (missing commas, empty sender) are now detected in parseEmailLine() and reported to stderr with the line number, instead of Gemini's original behavior of just skipping without any signal (

- Execution time

compareEmail() now takes const Email * instead of Email by value (Gemini's original compareEmails(Email a, Email b) copied both operands on every single comparison call, which happens $O(\log n)$ times per insert/extract)

Space complexity

Kept Gemini's single contiguous allocation and doubling realloc strategy

Maintainability

Split parsing out of main() into its own parseEmailLine() function with a clear 1/0 success contract, mirroring Gemini's original inline strtok block but now testable and reusable on its own.